

Profiling

Profiling is a practice that aims to extract and explain as much leakage from a device as possible.

Specifically, we know that Side Channel Analysis aims at exploiting noisy observations of the processing of an algorithm to recover its secret parameter.

Once the device to be attacked has been designated, if the adversary succeeds in appropriating a device that is very similar (if not the same), he can carry out operations aimed at profiling.

We can evaluate the efficiency of an attack like the minimum size of the attack set that is necessary to ensure that the attack succeeds in classifying the correct key among the first most probable ones.

Resources

To work on the project, I used:

https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1/ATM_AES_v1_fixed_key

The attack on github focuses on the output of the third S-box during the first round of the AES, targeting the dependency between the leakage measurements and the value output by the S-box.

The ASCAD database is structured in HDF5 files, with two main groups: 'Profiling_traces' and 'Attack_traces'.

HDF5 may be seen as a file system within a file, where files are datasets and folders are groups.

The group 'Profiling_traces' contains the data for the profiling phase (training) and 'Attack_traces' for the attack phase.

We note here that the traces are labelled with the output value of the S-box representing 256 possible classes.

This labelling allows each trace to be associated with the manipulated sensitive value, making the dataset usable for supervised machine learning techniques.

In addition to the traces and labels, the files also contain metadata such as plaintext, ciphertext, key and mask.

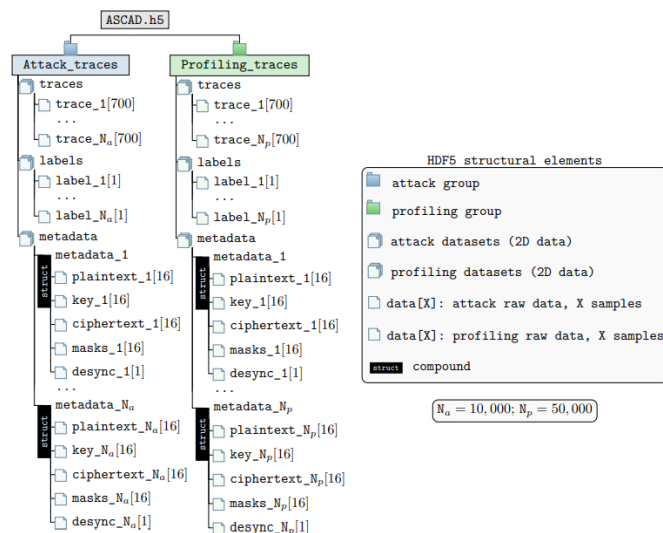


Figure 5: Structure of the ASCAD.h5 HDF5 data file

The side-channel observations obtained contain many tracks, each consisting of 100,000 time samples.

It should be noted that every trace in the final dataset that is generated by `ASCAD_generate.py` has been lightened.

This is indicated on page 11 in the paper: <https://eprint.iacr.org/2018/053.pdf>

It is used as a reference and quoted here: ‘https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1’.

Each trace has been reduced and now consists of 700 time samples, selected from the interval [45400..46100] of the original measurements.

The reasons for this interval are clarified by looking at the SNRs for the third Sub Byte operation of AES round 1.

This in fact turns out to be the target of our SCA.

In fact, evaluating the SNRs present in the various cases, it is possible to see peaks at the trace points in the interval [45400..46100].

Table 1: details of the SNR processings in Fig. 2

Name	Type	Definition of the target variable Z
<i>snr1</i>	unmasked sbox output	$\text{sbox}(p[3] \oplus k[3])$
<i>snr2</i>	masked sbox output	$\text{sbox}(p[3] \oplus k[3]) \oplus r_{\text{out}}$
<i>snr3</i>	common sbox output mask	r_{out}
<i>snr4</i>	masked sbox output in linear parts	$\text{sbox}(p[3] \oplus k[3]) \oplus r[3]$
<i>snr5</i>	sbox output mask in linear parts	$r[3]$

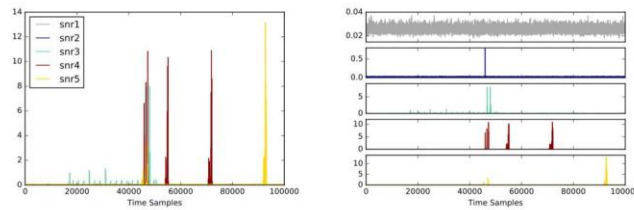


Figure 2: SNRs for various intermediate values related to the processing of $\text{sbox}(p[3] \oplus k[3])$.

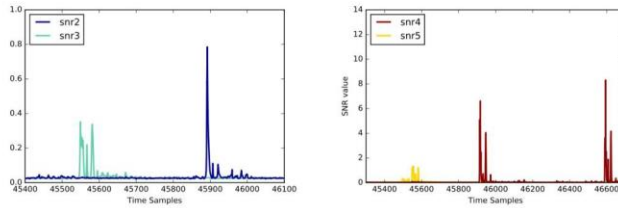


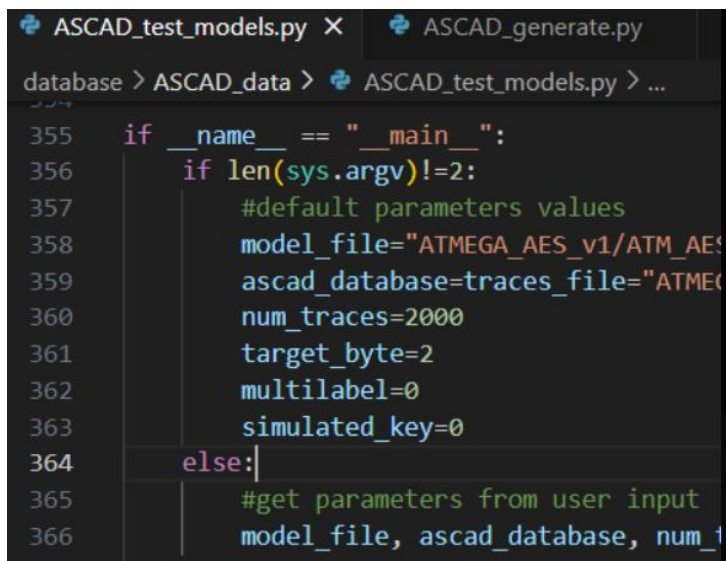
Figure 3: SNRs for various intermediate values related to the processing of $\text{sbox}(p[3] \oplus k[3])$ in the interval [45400..46100].

As already mentioned, they contain information concerning the calculation of the third S-box.

This is also specified as follows: 'target_byte : this is the index of the target byte during the attack. Default value is equal to 2 for ASCADv1 backward compatibility' at this link:

<https://github.com/ANSSI-FR/ASCAD/tree/master>

This can also be seen in the code, as in the test phase, only the third byte of the key is taken as the target of the attack:



```
ASCAD_test_models.py X ASCAD_generate.py
database > ASCAD_data > ASCAD_test_models.py > ...
355 if __name__ == "__main__":
356     if len(sys.argv)!=2:
357         #default parameters values
358         model_file="ATMEGA_AES_v1/ATM_AES
359         ascad_database=traces_file="ATMEC
360         num_traces=2000
361         target_byte=2
362         multilabel=0
363         simulated_key=0
364     else:
365         #get parameters from user input
366         model_file, ascad_database, num_t
```

Choice of Profiling Method

It is possible to evaluate different Classical or Machine/Deep Learning methods applicable to side-channel attacks.

Specifically, I decided to focus mainly on ML/DL approaches that are linked to the concepts of Multi-Layer Perceptrons (MLP) and Convolutional Neural Networks (CNN).

By running the code in `ASCAD_generate.py`, it is possible to generate these 3 files:

- **ASCAD.h5**
- **ASCAD_desync50.h5**
- **ASCAD_desync100.h5**

It was decided to simulate a more realistic situation, and to do this, a random delay of the tracks is inserted, resulting in `desync50` and `desync100`.

Choosing which model to train and which hyperparameters to use is by no means a trivial choice. You need to test which ones work best in the case of the problem you are trying to solve.

However, it is possible to follow a line of reasoning.

CNNs are composed of several layers with an MLP and a convolutional layer.

Due to the presence of the latter, in the case of working with misaligned traces, it is advisable to use a CNN model (which will take quite some time to train).

Conversely, if you choose to work with non-aligned tracks, then using an MLP model is very effective and trainable in a very short time (compared to that required by a CNN).

In the file `ASCAD_train_models.py` there are the functions `'mlp_best(...)'` and `'cnn_best(...)'` which proceed to create models as a result of tuning the hyperparameters.

Two main libraries are used within the code to carry out the training:

1. `tensorflow`
2. `tensorflow.keras.models`

I ran into some problems here due to version conflicts.

Apparently, tensorflow is not compatible with version 3.13 of Python on my computer. Note that the error refers to a file that is however present in my .venv created to contain the libraries needed for the code.

```
;59a676ad-09c4-42e8-943a-f9a62cdac23aTraceback (most recent call last):
  File "D:\Specialistica\Advanced Topics in Cybersecurity 2\Laboratorio 4 - Profiling\database\ASCAO_data\.venv\Lib\site-packages\tensorflow\python\pywrap_tensorflow.py", line 70, in <module>
    from tensorflow.python._pywrap_tensorflow_internal import *
ImportError: DLL load failed while importing _pywrap_tensorflow_internal: Impossibile trovare il modulo specificato.

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "D:\Specialistica\Advanced Topics in Cybersecurity 2\Laboratorio 4 - Profiling\database\ASCAO_data\ASCAO_train_models.py", line 20, in <module>
    import tensorflow as tf
  File "D:\Specialistica\Advanced Topics in Cybersecurity 2\Laboratorio 4 - Profiling\database\ASCAO_data\.venv\Lib\site-packages\tensorflow\_init_.py", line 40, in <module>
    from tensorflow.python import pywrap_tensorflow as _pywrap_tensorflow # pylint: disable=unused-import
  File "D:\Specialistica\Advanced Topics in Cybersecurity 2\Laboratorio 4 - Profiling\database\ASCAO_data\.venv\Lib\site-packages\tensorflow\python\pywrap_tensorflow.py", line 85, in <module>
    raise ImportError(
    ...<6 lines>...
    f'and include the entire stack trace above this error message.')
ImportError: Traceback (most recent call last):
  File "D:\Specialistica\Advanced Topics in Cybersecurity 2\Laboratorio 4 - Profiling\database\ASCAO_data\.venv\Lib\site-packages\tensorflow\python\pywrap_tensorflow.py", line 70, in <module>
    from tensorflow.python._pywrap_tensorflow_internal import *
ImportError: DLL load failed while importing _pywrap_tensorflow_internal: Impossibile trovare il modulo specificato.

Failed to load the native TensorFlow runtime.
See https://www.tensorflow.org/install/errors for some common causes and solutions.
If you need help, create an issue at https://github.com/tensorflow/tensorflow/issues and include the entire stack trace above this error message.
```

```
▼ LABORATORIO 4 - PROFILING
  ▼ database
    ▼ ASCAD_data
      ▼ .venv
        ▼ Lib\site-packages
          ▼ tensorflow
            ▼ python
              _pywrap_tensorflow_internal.lib
              _pywrap_tensorflow_internal.pyd
```

The solution I found was to work on another, less computationally powerful computer, on which I installed python version 3.9.

I also found it difficult to use the tensorflow.keras library in the ASCAD_train_models.py file. After enquiring, I chose to use the tf_keras library, which has very similar behaviour to the previous one.

Wanting to concentrate on the ML/DL aspects, I chose to first concentrate on working using the MLP model as it is trainable in a short time.

At this point, it is necessary to make the choice of the hyperparameters to be used. To do this, I made several attempts, trying to find the best compromise.

Important Observation

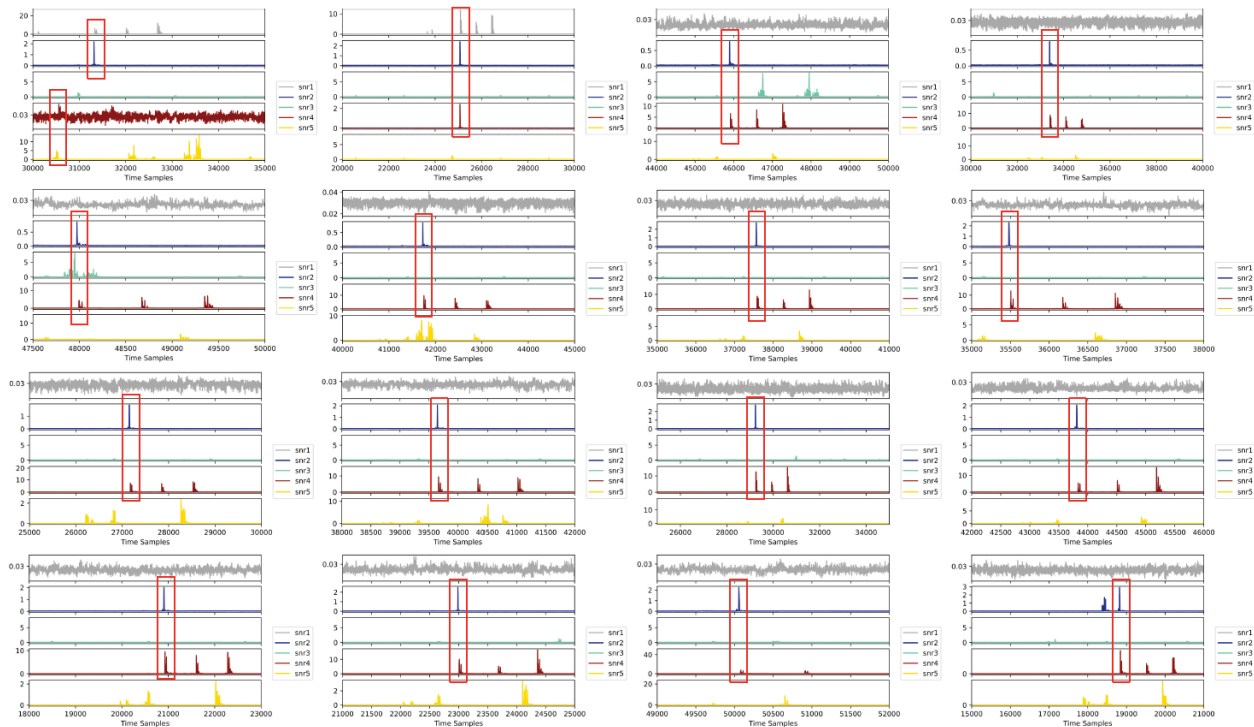
As mentioned earlier, this specific interval turns out to be very fruitful for trying to extrapolate leakage concerning the key of the third sub byte operation of first AES round.

So it might be a sensible idea to analyse the SNR relative to each sub byte operation of the first AES round, and then create 16 different databases.

Each is specific to one of the 16 bytes of the key, all of which will contain 100,000 traces (of which only 60,000 are used and split into 50,000 for profiling and 10,000 for attack). The difference is that all traces in a dataset will refer to the same interval, of 700 time samples, extrapolated from the total 100,000 time samples.

Which interval to use, depends on where the SNR is most significant regarding the sub byte operation we are interested in for that database.

To have an idea of that, inside the paper: **“Autoencoder Assist: An Efficient Profiling Attack on High-dimensional Datasets”** accepted by ICICS’22, we can see at page 22 this analysis.



They chose to use windows of 3000 samples (they called them Points of Interest)

Table 12: 3000 POIs trimmed for 16 S-boxes

S-box1 (28500,31500)	S-box2 (22500,25500)	S-box3 (44400,47400)	S-box4 (31930,34930)
S-box5 (46450,49450)	S-box6 (40230,43230)	S-box7 (36080,39080)	S-box8 (34000,37000)
S-box9 (25650,28650)	S-box10 (38170,41170)	S-box11 (27750,30750)	S-box12 (42330,45330)
S-box13 (19430,22430)	S-box14 (21500,24500)	S-box15 (48580,51580)	S-box16 (16970,19970)

Once obtained these intervals, it make sense to train 16 models on databases with that sample intervals for traces.

Each of these models will then be used singularly to execute a SCA for the specific byte of interest.

In this way, maybe it will be possible to make a successful attack.

Unfortunately, I only came to the realisation of this concept very late.

Moreover, it is not clear to me whether this could be the focus of this workshop.

In my project, I decided to use the dataset containing the traces composed of 700 time samples, extrapolated from [45400..46100] with respect to the 100,000 in the original case.

It makes sense for an MLP or CNN model to work correctly on finding the third byte of the key.

This is indeed clear in each of the various tests I have tried by varying the hyperparameters.

The associated rank is indeed 0 and the key is correctly identified.

What would happen if we try to obtain also others bytes key?

In other words, by performing the attack on all bytes using traces with 700 samples, is it possible to get other bytes of the key as well?

This seems to be a very ambitious challenge.

However, I chose to try using various hyper-parameters, creating various models that can be seen in the project folder.

Specifically, I chose to focus on an MLP network created using the `mlp_best` function in `ASCAD_train_models_ed.py`.

Exemples of training :

Model 1: MLP 250 EPOCHS 50 BATCH

Epoch 249/250

900/900 [=====] - 6s 7ms/step - loss: 5.2234 - accuracy: 0.0240 - val_loss: 5.5643 - val_accuracy: 0.0066

Epoch 250/250

900/900 [=====] - 7s 8ms/step - loss: 5.2218 - accuracy: 0.0240 - val_loss: 5.5696 - val_accuracy: 0.0054

Model 3: MLP 600 EPOCHS 32 BATCH:

Epoch 599/600

1407/1407 [=====] - 5s 3ms/step - loss: 4.8845 - accuracy: 0.0518 - val_loss: 5.8225 - val_accuracy: 0.0066

Epoch 600/600

1407/1407 [=====] - 5s 4ms/step - loss: 4.8831 - accuracy: 0.0514 - val_loss: 5.8090 - val_accuracy: 0.0080

Executing the attack

I used the trained models to attack all bytes.
These are the results I got.

	N. of train epoch	Batch	Byte found	Traces used to attack
Model 1	250	50	1/16	3000/10000
Model 2	500	50	2/16	3000/10000
Model 3	600	32	4/16	10000

So it was possible to recover even some other byte key than the one for the third seb byte operation.